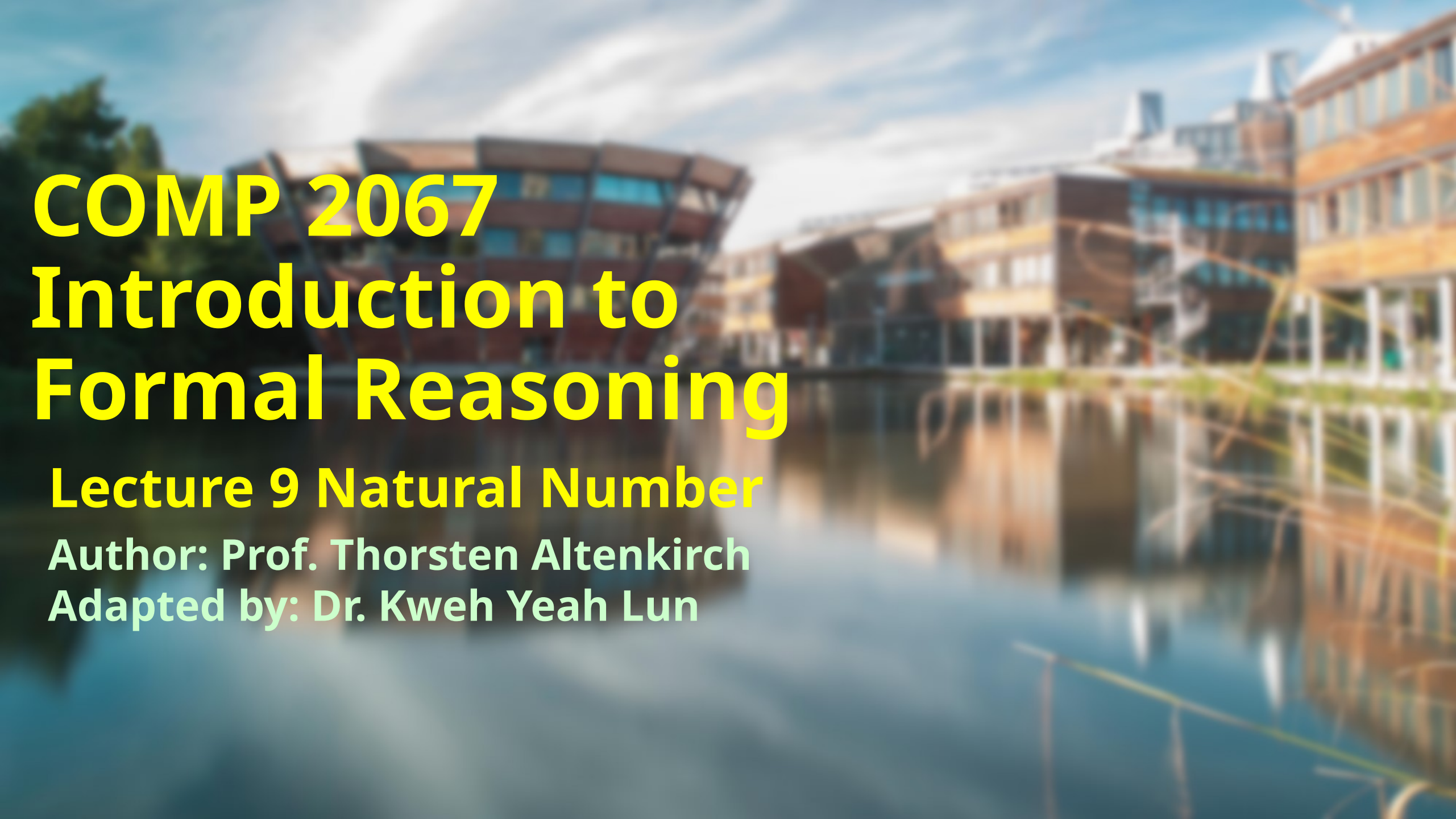

COMP 2067 Introduction to Formal Reasoning

Lecture 9 Natural Number

Author: Prof. Thorsten Altenkirch
Adapted by: Dr. Kweh Yeah Lun

University of Nottingham
UK | CHINA | MALAYSIA

Learning Outcomes

At the end of this lecture, you will be able to

- understand the inductive definition of natural number in Lean
- understand and prove some basic properties of natural number
- construct structural recursive steps for some operation of natural number
- apply tactic induction in Lean
- comprehend the definition of addition and multiplication using structural recursion
- understand the concept of decidability of natural number

University of Nottingham
UK | CHINA | MALAYSIA

Introduction

- Guiseppe Peano (1858 – 1932) codified the laws of natural numbers using predicate logic in the late 19th century.
- This is referred to as Peano Arithmetic.
- Peano viewed the **natural numbers** as **created from zero (0 = zero) and succ successor, i.e. 1 = succ 0, 2 = succ 1 and so on.**

- In Lean, this corresponds to the following **inductive definition**:

Try it!

```
inductive nat : Type
| zero : nat
| succ : nat → nat
```

- This declaration means:
  - There is a new type nat : Type,
  - There are two elements zero : nat, and given n : nat, then succ n : nat.
  - All the elements of nat can be generated by zero and then applying succ a finite number of times,
  - zero and succ n are different elements,
  - succ is injective, i.e. given succ m = succ n then, m = n.

University of Nottingham
UK | CHINA | MALAYSIA

Basic Properties of ℕ

- Lean adopt the notation that ℕ = nat
- Let's verify that 0 ≠ succ n which corresponds to true ≠ false for bool.
- Example:

```
example : ∀ n : ℕ, 0 ≠ succ n :=
begin
  assume n h,
  contradiction,
end
```

Try it!

University of Nottingham
UK | CHINA | MALAYSIA

- Next let's show that succ is injective.
- To do this, define a predecessor function pred using pattern matching which works the same way as for bool:

```
def pred : ℕ → ℕ
| zero := zero
| (succ n) := n

#reduce (pred 7)
```

Try it!

- Example:

```
theorem inj_succ : ∀ m n : nat, succ m = succ n → m = n :=
begin
  assume m n h,
  change pred (succ m) = pred (succ n),
  rewrite h,
end
```

Try it!

- Tactic **change**:
  - **to replace a subexpression within the goal or a hypothesis with a specified term or expression.**
  - rewrite parts of the goal or hypotheses, potentially simplifying the proof process or aligning it with a desired form.
  - In the example: replace the goal m = n with pred (succ m) = pred (succ n) exploiting that pred (succ x) = x

University of Nottingham
UK | CHINA | MALAYSIA

Structural recursion

- Example:

```
def double : ℕ → ℕ
| zero := 0
| (succ n) := succ (succ (double n))

#reduce (double 7)
```

Try it!

- double (succ n) is succ (succ (double n)).
- Every succ is replaced by two succ s.
- Example:

```
double 3
= double (succ 2)
= succ (succ (double 2))
= succ (succ (double (succ 1)))
= succ (succ (succ (succ (double 1))))
= succ (succ (succ (succ (double (succ 0)))))
= succ (succ (succ (succ (succ (succ (double 0))))))
= succ (succ (succ (succ (succ (succ (double zero))))))
= succ (succ (succ (succ (succ zero))))
= 6
```

- Example:

```
def half : ℕ → ℕ
| zero := zero
| (succ zero) := zero
| (succ (succ n)) := succ (half n)

#reduce (half 7)
#reduce (half (double 7))
```

Try it!

- half replaces every two succ s by one

```
half 7
= half (succ (succ 5))
= succ (half 5)
= succ (half (succ (succ 3)))
= succ (succ (half 3))
= succ (succ (half (succ (succ 1))))
= succ (succ (succ (half 1)))
= succ (succ (succ (half (succ zero))))
= succ (succ (succ zero))
= 3
```

University of Nottingham
UK | CHINA | MALAYSIA

Induction

- Example: prove that half is the inverse of double

[Try it!]

```
theorem half_double : ∀ n : ℕ , half (double n) = n :=
begin
  assume n,
  induction n with n' ih,
  refl,
  dsimp [double,half],
  apply congr_arg succ,
  exact ih,
end
```

- After assume n we are in the following st

```
n : ℕ
⊢ half (double n) = n
```

- Tactic **induction n**
  - induction n works very similar to cases, but it gives an extra assumption when proving the successor case, which labelled ih for induction hypoth
- After induction n, two subgoals were generated:

```
2 goals
case nat.zero
⊢ half (double 0) = 0

case nat.succ
n' : ℕ,
ih : half (double n') = n'
⊢ half (double (succ n')) = succ n'
```

- The first one is disposed off using refl, because half (double 0) = half 0 = 0.

```
n' : ℕ,
ih : half (double n') = n'
⊢ half (double (succ n')) = succ n'
```

- The successor case
- Here the extra assumption: succ n' holds for n’.
- Now apply the definitions of double and half using [double,half]:

```
n' : ℕ,
ih : half (double n') = n'
⊢ succ (half (double n')) = succ n'
```

half (double n'.succ)
= half (double (succ n'))
= half (succ (succ (double n')))
= succ (half (double n'))
= (half (double n')).succ

- Now use succ preserves equality by apply congr_arg succ:
- Then, exact ih

```
n' : ℕ,
ih : half (double n') = n'
⊢ half (double n') = n'
```

- Recall from predicate logic:

```
theorem congr_arg : ∀ f : A → B , ∀ x y : A, x = y → f x = f y
```

University of Nottingham
UK | CHINA | MALAYSIA

Addition

- Definition of addition also uses recursion.

Try it!

```
def add : ℕ → ℕ → ℕ
| m  zero     := m
| m  (succ n) := succ (add m n)
```

- m + n is defined as add m n.
- Example:

```
3 + 2
= add 3 2
= add 3 (succ 1)
= succ (add 3 1)
= succ (add 3 (succ 0))
= succ (succ (add 3 0))
= succ (succ (add 3 zero))
= succ (succ 3)
= 5
```

University of Nottingham
UK | CHINA | MALAYSIA

Multiplication

- Definition:

```
def mul : ℕ → ℕ → ℕ
| m 0     := 0
| m (succ n) := (mul m n) + m
```

Try it!

- x*y stands for mul x y.
- **As + was repeated succ, * is repeated +, that is m*n is m added n times**
- Example:

```
3 * 2
= mul 3 2
= mul 3 (succ 1)
= (mul 3 1) + 3
= (mul 3 (succ 0)) + 3
= (mul 3 0) + 3 + 3
= 0 + 3 + 3
= 6
```

University of Nottingham
UK | CHINA | MALAYSIA

Decidability

- Equality for natural numbers is decidable, that is they can be implemented as a function into bool.

Try it!

```
def eq_nat : ℕ → ℕ → bool
| zero     zero     := tt
| zero     (succ n) := false
| (succ m) zero     := false
| (succ m) (succ n) := eq_nat m n
```

- Example: show that eq_nat returns tt for equal numbers

Try it!

```
lemma eq_nat_ok_1 : ∀ n : ℕ , eq_nat n n = tt :=
begin
  assume n,
  induction n with n' ih,
  reflexivity,
  exact ih,
end
```

University of Nottingham
UK | CHINA | MALAYSIA

- Give yourself a big clap.
- Survived once again? Great! ⏪
- Any question?

University of Nottingham
UK | CHINA | MALAYSIA

Next lecture

- Lists

University of Nottingham
UK | CHINA | MALAYSIA

"The key to performance is elegance, not battling with the hardware."

– James Gosling

Hope you all enjoy this lecture!